

# Presenter Teleprompt — Simulation Run

15-minute slot · deck **s7-intake-animated-deck.html** + Control Centre, one screen, Cmd+Tab · requirement: **MapleSure-Disability-Claim-Submission-Requirement.pdf** · flow verified end to end on run **S7-00026** · present on the fresh, untouched run **S7-00027** (or create a new simulation run in the header on the day).  
**Read the large lines aloud. Small lines are actions — never read them.** → = press right-arrow on the deck (next beat / next slide).

---

## STAGE 1 · INTAKE — 0:00–3:30

SLIDE 1 · HOLD ~30S

The client asked for one thing: a business requirement carried all the way to production release, AI-assisted at every step. This is that journey — five stages — and at every stage you'll see the same pattern: **AI does the structuring, a named human makes the call.**

→ SLIDE 2 · HOLD ~15S

Stage one starts where every real project starts — with a document someone wrote in a hurry.

→ SLIDE 3 · BEAT 1 — THE DROP

A requirement arrives as a file. Nobody retypes it.

→ BEAT 2 — AI EXTRACTION

Title, business objective, numbered requirements — extracted from what was actually written, and labelled with exactly how it was produced.

→ BEAT 3 — AI ROUTING

Before spending anything on analysis, a cheaper question: does this fit an application we already know?

→ BEAT 4 — CLARIFICATION

And where the document is ambiguous, the analysis itself raises the questions — addressed to the business, not to a developer's guess.

⌘ → SWITCH TO APP — DIP 1 · INTAKE (~90S) · ROLE: BUSINESS OWNER

Intake → card 1. Source Requirement → upload **MapleSure-Disability-Claim-Submission-Requirement.pdf** → click **Extract**.

This is the real document — a disability claim submission requirement for our plan sponsor portal. And there's the extraction: title, objective, and seven numbered requirements, pulled from the document itself. In this environment that's a deterministic parser, and it says so on its face — **nothing here pretends to be AI.**

Point at the **Requirement Routing** verdict (already seeded).

**And the routing question is already answered: this fits the SponsorConnect repositories the run is grounded in.**

Click **Create Epic**, then sign the intake gate (follow the role picker).

**Two buttons, deliberately — no single role may both create the epic and sign the gate.**

⌘ → BACK TO DECK

## STAGE 2 · PLANNING — 3:30–6:30

→ SLIDE 4 · DIVIDER — ONE BREATH

**Stage two: one epic becomes a working plan.**

→ SLIDE 5 · BEAT 1 — GENERATE STORIES

**The epic breaks into stories with acceptance criteria, estimates, and a target repository each.**

→ BEAT 2 — DEPENDENCY MAP

**And a dependency map — which stories can run in parallel, which must wait.**

→ BEAT 3 — LIVE EDIT

**Edit a story and the map redraws. The plan is live, not a printout.**

→ SLIDE 6 · FOUR BEATS, ONE LINE EACH — BRISKLY

**Stories route by team... map onto repositories... roll up into a plan summary... and when a human edits any of it — **the app logs it**. Every edit is kept and labelled: AI-generated, human-edited. The provenance is the feature.**

Slide 7 ("the system learns") — SKIP if past 5:00. If shown, say the badge first: "This one is roadmap — target design, not today's build."

→ SLIDE 8 · BEAT 1 — PLAN SIGN-OFF

**Gate one. A named person, a required note, a locked plan.**

→ BEAT 2 — ARTIFACTS

**The signed plan renders into portable artifacts — plain files, the language agents and humans both read.**

→ BEAT 3 — PUSHED TO GIT

**...delivered to each team's own repository, on a disposable branch a human chooses to merge.**

⌘ → SWITCH TO APP — DIP 2 · PLAN & SIGN-OFF (~90S)

Planning → Epic to Stories → ♦ AI Suggest Stories → scroll the seven stories.

**Seven stories — acceptance criteria, estimates, teams, repositories.**

Open one story → Edit Story → tweak the title → Save Changes.

**Tracked: AI Generated, Human Edited.**

Planning → Plan Sign-off → walk the green ticks → enter approver + note → 🔒 Approve Plan & Lock.

**From this exact version, everything downstream traces back. And note what Gate one does **not** authorise — the AI still hasn't written a line of production code.**

⌘ → BACK TO DECK

## STAGE 3 · BUILD & REVIEW — 6:30–11:00

→ SLIDE 9 · DIVIDER

**Stage three is where most AI demos go quiet. Ours gets louder — because S7 is the governed control plane, not an IDE. **Humans build, in their own tools.** S7 briefs them, watches the evidence, and reviews independently.**

→ SLIDE 10 · FOUR BEATS — ONE LINE EACH, LAND ON BEAT 4

**The Architecture Agent wakes... the application, mapped... external systems and the facts... and a human accepts it. Generated by the service, accepted by the Engineering Lead. **No phase in this system approves its own output** — you'll hear that sentence again.**

→ SLIDE 11 · THREE BEATS — BRISKLY

**Three teams, three briefings — each pack references the signed plan and architecture by version, never copies them — delivered into each team's own repo.**

→ SLIDE 12 · BEAT 1 — THE BRIEFING ARRIVES

**The developer's world: their IDE, their Git. Human controlled, AI assisted.**

→ BEAT 2 — THE DASHBOARD FOLLOWS

**The dashboard follows by itself — from commits, PRs and CI, not from status meetings.**

→ BEAT 3 — THE NEXT STORY UNLOCKS

**Stories unlock in dependency order, on evidence.**

→ SLIDE 13 · BEATS 1-2 — QUICKLY

**Tests run in the workspace; the checklist ticks itself — one row per acceptance criterion.**

→ BEAT 3 — A FAILURE BLOCKS THE DOOR

**And when something fails, it blocks. Let me show you exactly that — a failure, live.**

⌘ → SWITCH TO APP — DIP 3 · THE BUILD STORYLINE (~2:30) — THE CENTREPIECE

**Build & Review** → **Architecture** → **Generate Architecture** → one scroll of the preview → **Accept Architecture** (role: Engineering Lead, enter a name).

**Generated in seconds — accepted by a named human.**

**Generate Delivery Packs** → open one pack → show AGENTS.md + test plan → **Approve Test Plan** on each pack (role: QA Lead) → **Publish All** (role: Engineering Lead).

**Every pack carries a test plan — one deliberately failing test per acceptance criterion, approved by QA before anything publishes. And publication only ever writes to a fresh s7 branch — it refuses the default branch.**

**Developer Workspaces** → **Sync from Git**. (US-001 goes green.)

**Real signal: commit, PR, CI. US-001 is done.**

**Sync from Git** again. (US-002 green.)

**Same again — so far, the happy path.**

**Sync from Git — US-003 arrives RED: its git push failed. STOP.** Let the room look at it.

**This is the honest moment. The sync button doesn't editorialize — bad news lands on the dashboard exactly as fast as good news, and this story is now blocked. Nothing downstream of it moves.**

Click **Rerun US-003 sync** → push succeeds, story green.

**A human retried it, deliberately. Recovery is an action someone takes and the record keeps — not something the system quietly papers over.**

**Sync from Git**. (US-004 + US-005 land together.)

**Two stories at once — parallel iteration, exactly as the dependency map planned it.**

**Sync from Git** once more. (US-006 + US-007 complete the board.)

**And the board completes. Everything you're seeing is simulated end to end — and every artifact says so on its face. That labelling discipline is the product.**

⌘ → BACK TO DECK

→ SLIDE 14 · THREE BEATS — ONE PASS

**Independent review: separated duties, evidence-based — and the verdict is recorded. A blocked review is never overwritten; the passing one is a new version underneath it.**

→ SLIDE 15 · HOLD 10S

**One view: done, moving, blocked — and why.**

## STAGE 4 · FINAL GATING — 11:00–12:30

→ SLIDE 16 · DIVIDER

**Stage four: before release, independent inspection.**

→ SLIDE 17 · BEAT 1 — THE INSPECTION AGENTS WAKE

**Checks run across every story's evidence.**

→ BEAT 2 — DRIFT FOUND, AND GOVERNED

**Findings become named conditions — never a single magic score.**

→ BEAT 3 — A HUMAN PASSES THE GATE

**And a QA Lead decides. Same sentence as before: **no phase approves itself.****

⌘ → SWITCH TO APP — DIP 4 · QUALITY GATE (~45S) · ROLE: QA LEAD

**Final Gating** → **Run quality checks** → walk two rows of the evidence table (Check / Status / Evidence / Owner).

**Every check names its evidence and its owner. And where a check reads "not applicable" — a system that says that out loud is a system whose "passed" means something.**

**Decide final gate.**

⌘ → BACK TO DECK

## STAGE 5 · RELEASE — 12:30–15:00

→ SLIDE 18 · DIVIDER

**Stage five. The button a human presses.**

→ SLIDE 19 · BEAT 1 — RELEASE READINESS

**Everything is already proven by the time this page matters.**

→ BEAT 2 — A HUMAN APPROVES

**Five named roles, five separate signatures — not one signature standing in for a team.**

→ BEAT 3 — RELEASED, WITH ITS AUDIT TRAIL

**And the release ships with its own history attached.**

⌘ → SWITCH TO APP — DIP 5 · RELEASE (~60S)

**Release** page → sign two roles via the header role picker (Business Owner → QA Lead), name + note each.

**In the interest of time I'll sign two of the five — each row gets its own name and timestamp.**

Release document card → generate → open the download.

**The run renders its own record — every approval, every acceptance criterion with its result, every review verdict. A deterministic rendering of what actually happened, and badged as exactly that.**

⌘ → BACK TO DECK · → SLIDE 20 — LET THE TITLE SIT, THEN CLOSE

**Every artifact you saw told you what it was — AI-generated, rule-based, human-edited, or simulated — on its own face. **That's the product.** Not that AI writes the plan: that you can always tell which parts it wrote, and a named human signed every gate between the requirement and production.**

**If anything breaks:** switch the header run selector to **S7-00026** — the same flow, already completed end to end — point instead of clicking, and keep talking. **Lost on the deck:** the footer counter shows the slide; Home restarts, End jumps to the takeaway.